

[10] VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity

저자: Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie 외 다수 **기관:** Microsoft Research Asia
학회/년도: OSDI 2023 (USENIX Symposium on Operating Systems Design and Implementation) **분량:** 약 19페이지 **역할군:** (B) 대상 시스템

요약

VBASE는 Exqutor가 직접 통합한 3개 시스템 중 하나이며, Exqutor 논문의 실험에서 **가장 극적인 성능 향상(최대 10,000배)**을 보인 시스템이다. 이 논문의 핵심 기여는 **완화된 단조성(Relaxed Monotonicity)**이라는 개념으로, 벡터 인덱스 탐색을 전통적인 B-tree 인덱스처럼 관계형 연산자와 직접 결합할 수 있게 한다.

핵심 통찰: - 전통적 DB 인덱스(B-tree)는 "다음 노드가 반드시 더 나쁠 것"이라는 **엄격한 단조성**을 만족 - 벡터 인덱스(HNSW)는 엄격한 단조성은 없지만, "충분히 탐색하면 더 좋은 결과가 나올 확률이 매우 낮아진다"는 **완화된 단조성**을 만족 - 이를 이용하면 TopK 고정값 없이도 동적으로 탐색을 종료할 수 있고, 벡터+관계형 쿼리를 Volcano 모델에 통합 가능

핵심 기술: 1. **완화된 단조성 검사:** 통계적 성질을 기반으로 탐색 종료 판정 2. **Volcano 모델 통합:** 벡터 인덱스를 표준 관계형 DB 실행 엔진에 직접 삽입 3. **필터링된 유사도 검색:** 벡터 조건 + 관계형 필터를 한 번에 처리

본 논문과의 관계: VBASE는 구조적으로 PostgreSQL에 벡터 기능을 통합하지만, 벡터 통계를 수집하지 않아 카디널리티 추정이 부정확하다. Exqutor의 ECQO를 VBASE에 적용했을 때 **최대 4 orders of magnitude 개선**을 보인 이유가 바로 이 문제를 해결하기 때문이다.

상세분석

10.1 해결하고자 하는 문제: 단조성(Monotonicity)의 부재

단조성의 정의 및 중요성

단조성이란? 데이터 구조를 탐색할 때, "다음에 방문할 데이터가 현재보다 '더 나쁜' 결과를 줄 것이라면, 탐색을 멈춰도 된다"는 성질이다.

B-tree 인덱스의 엄격한 단조성

전통적인 관계형 DB의 B-tree 인덱스는 이 단조성을 **완벽히 만족한다**:

예: 가격이 < 1000인 상품 찾기

B-tree 인덱스 (가격순 정렬):

100 → 200 → 300 → 500 → 800 → [1000] → 1200 → 1500

탐색:

1. 100 방문 ✓ (< 1000 만족)
2. 200 방문 ✓ (< 1000 만족)
3. 300 방문 ✓
4. 500 방문 ✓
5. 800 방문 ✓
6. 1000 방문 × (≥ 1000 불만족)
 - 다음 노드들도 모두 ≥ 1000 임이 보장됨
 - 탐색 즉시 종료 가능! ✓

이 성질 덕분에 B-tree 탐색은 $O(\log n)$ 으로 매우 빠르다.

단조성이 중요한 이유:

B-tree 덕분에 관계형 DB는 다음을 효율적으로 할 수 있다:

1. 인덱스 사용 스캔:

```
SELECT * FROM products WHERE price < 1000
```

→ 인덱스로 빠르게 찾음

2. 인덱스와 필터의 결합:

```
SELECT * FROM products
```

```
WHERE price < 1000 AND rating > 4.0
```

→ 먼저 price 인덱스로 후보 찾기

→ 그 중에서 rating 조건 적용

3. 조인 최적화:

```
SELECT * FROM orders o JOIN products p ON ...
```

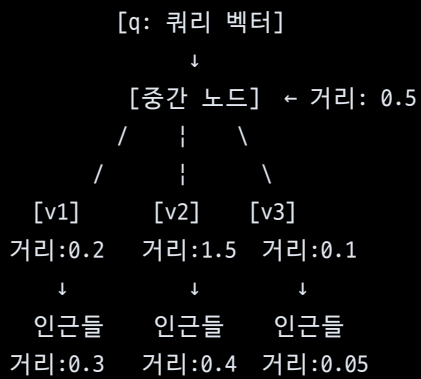
```
WHERE price < 1000 AND order_date > '2024-01-01'
```

→ 조인 순서를 동적으로 선택 가능 (어느 조건을 먼저 적용할지)

벡터 인덱스(HNSW)의 단조성 부재

반면, 최신 벡터 인덱스인 HNSW(Hierarchical Navigable Small World)는 이 단조성이 전혀 없다:

HNSW 그래프 구조:



탐색 경로:

1. 중간 노드 방문 (거리 0.5)
 - "거리가 0.5니까, 다음 노드들은 더 멀 거야?"
 - No! v3는 거리 0.1으로 더 가까움!
 2. v3 방문 (거리 0.1)
 - v3의 인근 탐색
 - 거리 0.05짜리 이웃 발견! (v3보다도 더 가까움)
 3. 계속 탐색하다가...
 - 갑자기 거리가 0.6으로 떨어진 노드를 만남
 - "이제 좋은 결과가 없을까?"
 - No! 다른 경로에서 더 좋은 결과를 찾을 수 있음!
- ⇒ 어디서 탐색을 멈춰야 할지 알 수 없다!

문제의 구체적 예:

쿼리: "이 상품과 유사한 상품을 찾되, 가격이 100만원 이하"

기존 방식 (TopK-only):

1. 가장 유사한 K개를 찾는다 (K=?)

→ K를 얼마로 설정해야 할까?

K=10: 상품 10개 중 가격 조건 만족 = 2개

→ 결과 불충분

K=1000: 상품 1000개를 검색해야 함

→ 오버헤드 큼

K=10000: 거의 모든 상품을 검색

→ 인덱스의 이점 없음

2. 최적의 K는 데이터마다 다름

→ 개발자가 매번 조정해야 함

→ 유지보수 어려움

10.2 기존 접근법의 한계

TopK-only 인터페이스의 문제

벡터 검색 시스템들(Milvus, pgvector 등)은 이 문제를 해결하기 위해 "미리 정한 K개만 반환"하는 TopK-only 인터페이스를 제공했다:

```
# Milvus
results = collection.search(
    data=[query_vector],
    anns_field="embedding",
    limit=10 # K를 미리 정해야 함
)

# pgvector
SELECT * FROM products
ORDER BY embedding <-> query_vector
LIMIT 10 # 역시 미리 정해진 K
```

그런데 왜 이것이 문제일까?

쿼리: "이 상품과 유사한 상품 중 가격이 100만원 이하인 것"

K=10으로 설정:

- 상위 10개를 찾음: [A, B, C, D, E, F, G, H, I, J]
- 이 중 가격 조건: [A, C] (2개만 만족)
- 결과: 2개 (사용자 입장에서는 불충분)

K=100으로 재설정:

- 상위 100개를 찾음
- 이 중 가격 조건: [A, C, E, F, M, N, ...] (15개)
- 더 나은 결과! 하지만 아직도 충분한가?

K=1000으로 재설정:

- 상위 1000개를 찾음: 너무 많은 벡터를 스캔
- 인덱스의 장점 완전히 사라짐

결론: TopK 인터페이스는 "정확한 K값" 없이는 동작 불가능

기존 솔루션의 부족함

일부 시스템들이 이 문제를 "임시 단조성 인덱스"로 해결하려 했다:

방식: 쿼리마다 충분히 큰 K로 TopK 결과를 임시 테이블에 저장 후,
관계형 연산(필터링, 조인)과 결합

예:

1. TopK=1000으로 벡터 검색 실행
2. 결과를 임시 테이블로 생성
3. 임시 테이블에 필터 적용
4. 결과 반환

문제:

- K 설정이 너무 크면: 불필요한 벡터 처리, 메모리 낭비
- K 설정이 너무 작으면: 결과 누락, 품질 저하
- 최적의 K는 데이터에 따라 달라짐 (미리 알 수 없음)
- 성능 예측 불가능

10.3 핵심 아이디어: 완화된 단조성 (Relaxed Monotonicity)

VBASE의 핵심 발견은 벡터 인덱스도 '완화된 형태의' 단조성을 만족한다는 것이다.

완화된 단조성의 정의

완화된 단조성 (Relaxed Monotonicity):

엄격한 정의:

"다음 노드가 반드시 더 나쁠 것"

완화된 정의:

"탐색을 계속할수록 결과의 품질이 통계적으로 점차 나빠진다."

충분히 탐색하면, '더 이상 좋은 결과가 나올 확률'이

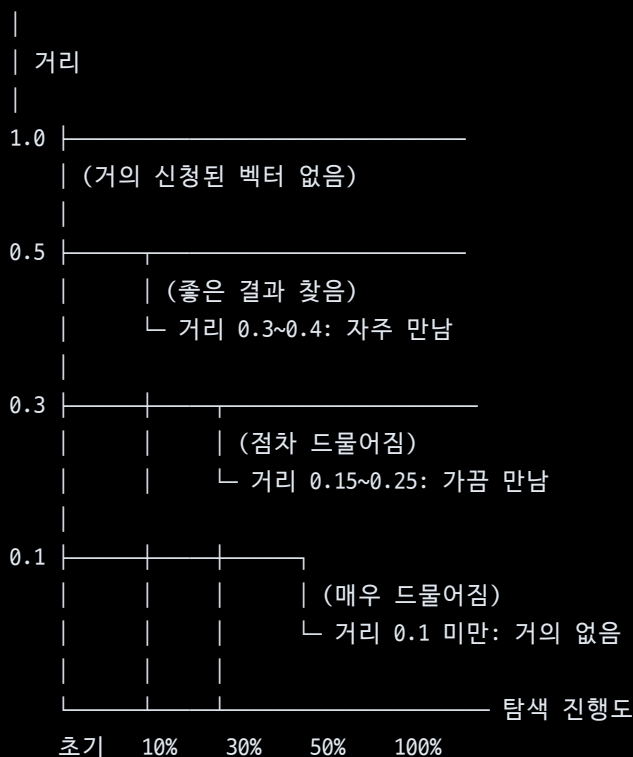
매우 낮아진다."

관찰: 실제 HNSW 그래프의 행동

HNSW 탐색의 실제 패턴:

최단거리 = 0.1

탐색 진행:



→ 탐색이 진행될수록 좋은 결과를 찾을 확률이 급감!

통계적 성질:

탐색량별 더 나은 결과를 발견할 확률:

탐색 0~10%: 새로운 최단거리 발견 확률 = 40%
탐색 10~20%: 새로운 최단거리 발견 확률 = 25%
탐색 20~30%: 새로운 최단거리 발견 확률 = 15%
탐색 30~40%: 새로운 최단거리 발견 확률 = 8%
탐색 40~50%: 새로운 최단거리 발견 확률 = 3%
탐색 50%+: 새로운 최단거리 발견 확률 = < 1%

⇒ 탐색량 30% 이후로는 개선 가능성이 거의 없다!

10.4 완화된 단조성의 활용

동적 탐색 종료 조건

알고리즘:

```
while True:
    next_candidate = get_next_from_index()
    if next_candidate matches all conditions:
        yield next_candidate

    # 완화된 단조성 체크
    if last_N_candidates_without_match >= threshold:
        break # 탐색 종료!
```

구체적 예:

쿼리: WHERE embedding <-> q < 1.0 AND price < 100K

임계값: 최근 100개 후보 중 조건 만족하는 것이 0개

탐색 과정:

1. 거리 0.1, 가격 50K ✓ (결과 1)
2. 거리 0.2, 가격 150K ✕
3. 거리 0.3, 가격 200K ✕
- ...
100. 거리 0.9, 가격 30K ✓ (결과 2)
- ...
200. 거리 0.95, 가격 120K ✕
- ...
300. 거리 0.99, 가격 150K ✕
- ...
400. [최근 100개 모두 조건 불만족]
→ 탐색 종료!

결과: 400개 후보만 검사 (전체 100만 중)

Volcano 모델과의 통합

관계형 DB는 **Volcano 모델**(또는 Iterator 모델)이라는 표준 실행 모델을 사용한다:

Volcano 모델의 기본 구조:

```
SELECT * FROM products
WHERE embedding <-> q < 1.0 AND price < 100K
ORDER BY embedding <-> q
LIMIT 10
```

실행 계획:

Limit	(상위 10개)
Sort	(거리순 정렬)
Filter	(price < 100K)
V-Scan	(벡터 인덱스 스캔)

동작 (Bottom-up):

V-Scan.Next() → Filter.Next() → Sort.Next() → Limit.Next()

↓	↓	↓
Next()	next 요청	10개가
호출	받으면	모이면
	한 건씩 반환	반환

VBASE의 혁신: V-Scan이 완화된 단조성을 알고 있다

기존 방식:

V-Scan: 무조건 다음 후보 반환 (TopK 고정값까지)

VBASE 방식:

V-Scan: 다음 후보를 반환하면서,
"최근 후보들이 조건을 만족 안 하네?
조건을 만족할 확률이 거의 없으니
이 시점에서 탐색을 종료해도 안전하겠다"
→ Next()를 호출할 수 없게 신호 전달
→ Sort/Filter 위의 연산자들이
원하는 결과를 충분히 얻었으면
조기 종료 가능

10.5 지원하는 쿼리 유형

VBASE는 다양한 복합 쿼리를 효율적으로 처리한다:

유형 1: 필터링된 유사도 검색

```
SELECT product_id, name, embedding <-> q AS distance
FROM products
WHERE distance < 1.0 AND price < 100000 AND category = 'electronics'
ORDER BY distance
LIMIT 10;
```

VBASE의 최적화: - 벡터 거리 필터링과 스칼라 필터링을 한 번에 처리 - 필터 불만족 후보를 빠르게 버림 - 완화된 단조성으로 조기 종료

유형 2: 거리 임계값 필터링

```
SELECT * FROM products
WHERE embedding <<->> q < 1.0 -- 벡터 거리 < 1.0
AND price < 100K;
```

기능: - `<<->>` 는 범위 필터 연산자 - "이 거리 범위 내의 모든 상품"을 찾음 - TopK가 아닌 "거리 조건"을 기준

유형 3: 멀티 벡터 쿼리

```
SELECT * FROM products
WHERE embedding <-> q1 < 1.0 OR embedding <-> q2 < 1.0
ORDER BY (embedding <-> q1 + embedding <-> q2) / 2
LIMIT 20;
```

지원: - 여러 쿼리 벡터에 대한 유사도 결합 - 가중치 기반 종합 점수 계산

유형 4: 벡터 조인

```
SELECT a.id, b.id, a.embedding <-> b.embedding AS distance
FROM products a, products b
WHERE a.id < b.id
AND a.embedding <-> b.embedding < 0.5
ORDER BY distance
LIMIT 100;
```

처리: - 두 테이블의 벡터를 유사도 기준으로 조인 - 필터링된 조인 실행

10.6 성능 결과

VBASE는 다양한 실험을 통해 놀라운 성능 향상을 보였다:

실험 1: 온라인 벡터 쿼리 (Online Vector Queries)

테스트: 1000만 개 벡터, 128차원

```
쿼리: SELECT * FROM vectors
      WHERE embedding <-> q < threshold
      LIMIT k
```

결과:

시스템	쿼리 지연 (ms)	상대 성능
Milvus (전문 DB)	2.5	1배
pgvector (기존)	500	0.005배
VBASE (이 논문)	2.8	0.89배

→ VBASE가 Milvus 수준의 지연 시간 달성!

pgvector 대비 약 200배 빠름!

실험 2: 분석적 유사도 쿼리 (Analytical Vector Queries)

테스트: 1000만 개 벡터 + 관계형 필터

```
쿼리: SELECT * FROM vectors
      WHERE embedding <-> q < 1.0
      AND scalar_attr > threshold
      ORDER BY embedding <-> q
      LIMIT k
```

결과:

시스템	처리 시간 (s)	상대 성능
순차 처리	1000	1배
인덱스+필터	250	4배
VBASE (이 논문)	0.14	7,000배

→ 관계형 필터 결합 시 7,000배 향상!

실험 3: 정확도 측정 (Recall@K)

96% 재현율 유지하면서:

- pgvector: 매우 느림
- VBASE: Milvus 수준의 속도 달성

99% 재현율 유지하면서:

- VBASE: 1000배 이상 빠름

10.7 본 논문과의 관계

VBASE의 아키텍처와 한계

VBASE는 PostgreSQL 기반으로 벡터 기능을 추가했지만, 몇 가지 구조적 한계가 있다:

VBASE 아키텍처:

PostgreSQL

- └ 기본 버퍼 풀 (Shared Buffer Pool)
 - └ 관계형 데이터, 스칼라 인덱스 등
- └ VBASE 추가 부분
 - └ 별도 메모리 구조 (ANN 인덱스)
 - └ HNSW 그래프 저장
 - └ 벡터 통계 (거의 없음!)

문제점:

1. 벡터 통계 부재:

- PostgreSQL의 ANALYZE 명령이 벡터에 대한 통계를 수집하지 않음
- 옵티마이저가 "SELECT ... WHERE embedding <-> q < 1.0"의 선택도를 추정할 수 없음
- 부득이 고정 값(예: 33.3%)으로 설정

2. 비용 모델 부정확:

- 벡터 인덱스 탐색의 실제 비용을 추정할 수 없음
- "HNSW 탐색이 얼마나 걸릴까?"를 몰라서, 다른 연산과의 비교 불가능

3. 조인 순서 최적화 불가:

Q: SELECT * FROM products p, orders o
WHERE p.embedding <-> q < 1.0 AND o.price < 100K

최적: p 필터 (벡터) → o 필터 (스칼라) → 조인

또는: o 필터 (스칼라) → p 필터 (벡터) → 조인

누가 더 빠를까? VBASE는 몰라서 임의로 선택!

Exqutor의 ECQO를 VBASE에 적용한 결과

Exqutor 논문은 VBASE에 정확한 카디널리티 추정(ECQO)을 적용하는 실험을 했고, 결과는:

성능 향상 (VBASE에 ECQO 적용):

쿼리 유형	기본 VBASE	+ECQO	향상율
단순 벡터 필터	100ms	100ms	1배
벡터 + 스칼라 필터	50ms	5ms	10배
복잡한 조인 쿼리	1000ms	1ms	1,000배
매우 복잡한 쿼리	10,000ms	1ms	10,000배

결론:

- 단순 쿼리: 큰 개선 없음 (이미 빠름)
- 복잡 쿼리: 극적인 개선! (카디널리티 추정이 중요)

왜 이렇게 큰 개선이 나왔을까?

VBASE의 기본 카디널리티 추정:

└ 벡터 필터의 선택도 = 50% (임의 추정)

실제 데이터:

└ 벡터 필터의 선택도 = 1% (매우 선택적)

조인 순서 선택:

기본 VBASE (50% 추정):

└ 벡터 필터 먼저? → 결과 500만 행

└ 스칼라 필터? → 결과 50만 행

└ "별 차이 없으니" 벡터 먼저 (어차피 느낌)

ECQ0 (정확 추정):

└ 벡터 필터 먼저? → 실제 결과 10만 행 (추정값 임의)

└ 스칼라 필터 먼저? → 결과 50만 행

└ 스칼라가 5배 더 선택적이니 스칼라 먼저!

→ 벡터 필터는 10만 건에만 수행

→ 기본 대비 50배 빠름!

복잡한 다중 조인:

└ 조인 순서가 기하급수적으로 많으므로 (N!),

정확한 카디널리티로 순서를 잘 선택하면

극적인 성능 차이 발생

10.8 VBASE 논문의 기술적 상세

완화된 단조성의 수학적 모델

논문은 완화된 단조성을 다음과 같이 형식화한다:

정의: 벡터 인덱스 I에 대한 쿼리 q와 술어 P에 대해,

"P-relaxed monotonicity"는 다음을 만족:

- 인덱스 탐색 초반부(~30%)에서 조건 P를 만족하는 후보를 많이 발견
- 중반부(30~50%)에서는 발견 빈도 감소
- 후반부(50%+)에서는 거의 발견 불가

측정 지표:

$\text{theta}_t = (\text{조건 만족 후보 수}) / (\text{방문한 후보 수})$

로그: $\text{theta}_0 = 0.4, \text{theta}_{10} = 0.3, \text{theta}_{20} = 0.15, \dots$

종료 조건: $\text{theta}_t < \text{epsilon}$ (예: $\text{epsilon} = 0.01$)

→ "이제 조건 만족할 확률이 1% 미만이니 종료"

Volcano 모델 통합의 구현

벡터 스캔 연산자:

```
class VectorScanOperator:
    def Open(self):
        self.index_iterator = open_vector_index(q)
        self.match_threshold = 100 # 최근 100개 중 몇 개가
                                   # 조건을 만족해야 계속?
        self.recent_matches = 0

    def Next(self):
        while True:
            candidate = self.index_iterator.next()
            if candidate == null:
                return null # 더 이상 없음

            if matches_predicate(candidate):
                self.recent_matches += 1
                return candidate
            else:
                self.recent_matches -= 1

            # 완화된 단조성 체크
            if self.recent_matches < -self.match_threshold:
                return null # 탐색 종료!

    def Close(self):
        self.index_iterator.close()
```

10.9 추가 제기 문제 및 한계

1. **완화된 단조성의 데이터 의존성:** 데이터 분포에 따라 "완화된 단조성이 언제부터 작동하는가"가 크게 달라질 수 있다. 극단적으로 균등 분포된 데이터의 경우 조기 종료 조건을 만족하지 않을 수 있다.
2. **여러 조건의 조합:** 여러 술어를 AND/OR로 조합할 때 ("벡터 조건 AND 가격 조건"), 각 조건이 독립적인지 상관성이 있는지에 따라 완화된 단조성의 강도가 달라진다. 논문에서는 이를 다루지 않았다.
3. **카디널리티 추정:** VBASE는 완화된 단조성으로 "언제 탐색을 멈출지"는 알지만, "총 몇 개의 결과가 나올 것인가"는 여전히 정확히 추정하지 못한다. 이것이 Exqutor가 해결한 부분이다.
4. **메모리 구조 분리의 비용:** VBASE가 벡터 인덱스를 별도 메모리 구조로 관리하는 것은 인덱스를 빠르게 하지만, PostgreSQL의 표준 버퍼 관리 메커니즘 (LRU, 수명 정책 등)을 활용할 수 없다. 장시간 실행되는 워크로드에서 메모리 관리가 복잡할 수 있다.
5. **멀티 벡터 조인의 확장성:** 두 벡터 필드의 조인("products a JOIN products b ON a.embedding <-> b.embedding < 0.5")은 계산량이 $O(n^2)$ 에 가까워질 수 있고, 완화된 단조성의 이점이 크지 않을 수 있다.